> *Note* — The W3C has an interest group note that discusses several ways of achieving this URI/URL mapping, including how a single webpage can describe multiple tags using either HTTP redirects (which requires some server setup) or what they call “Hash URIs” (which require no setup).
>
> That interest group note also explains why it might be desirable to have a separate URIs for a concept and the document describing that concept. Because of the structure of the schema, that separation is less important for FamilySearch GEDCOM 7 than it is for the semantic web, but it remains good advice where feasible.

The schema structure may contain the same tag more than once with different URIs. Reusing tags in this way must not be done unless the concepts identified by those URIs cannot appear in the same place in a dataset, and should not be done unless the URIs identify closely related concepts.

> *Example* — Consider three extensions:
>
> - `https://example.com/LocationRecord`, a record that describes a location.
> - `https://example.com/LocationPointer`, a substructure of most events that points to a `https://example.com/LocationRecord`.
> - `https://example.com/inLocoParentis`, a substructure of some events indicating a non-parent entity that filled the legal role of a parent for that event.
>
> Given this, we have the following:
>
> - `https://example.com/LocationPointer` and `https://example.com/inLocoParentis` must not be given the same tag because they can appear in the same place in a dataset.
> - `https://example.com/LocationRecord` and `https://example.com/inLocoParentis` may be given the same tag, but should not be given the same tag because they identify unrelated concepts.
> - `https://example.com/LocationRecord` and `https://example.com/LocationStructure` may be given the same tag.
>
> One way to satisfy these constraints and recommendations is with the following schema:
>
> ```
> 1 SCHMA
> 2 TAG _LOC https://example.com/LocationRecord
> 2 TAG _LOC https://example.com/LocationPointer
> 2 TAG _ILP https://example.com/inLocoParentis
> ```

An extension tag that is not given a URI in the schema structure is called an **undocumented extension tag**. The meaning of an undocumented extension tag is identified by its superstructure type and its tag.

1.5.2. Requirements and Recommendations

- It is recommended that applications not use undocumented extension tags.

- It is required that each tag definition's extension tag be unique within the document.
- It is recommended that each documented extension tag's URI be unique within the document.
- It is recommended that extension creators use URLs as their URIs and serve a YAML file describing the meaning of an extension at its URL, such as a file in the GEDCOM structure registry. The FamilySearch GEDCOM YAML file format defines how the YAML file provides machine-readable documentation and metadata about extensions. For example, the `superstructures` key in the YAML file can be used to disambiguate which meaning applies within a given context, such as in the `_LOC` example above where the meaning is context-dependent.

1.5.3. Extension versus Standard

Standard structures take priority over extensions. Data contained in extension tags will not be interpreted by other systems correctly unless the other system supports that particular extension. In particular, those supporting extensions should keep in mind the following:

- If a standard structure is present that contradicts an extension that is present, the standard structure has priority and the extension should be updated to align with it.

> *Example* — If a document has an extension `_ISODATE` in ISO 8601 format that disagrees with a `DATE` (p.69) in the `DateValue` (p.25) format, the `DATE` shall be taken as more correct and the `_ISODATE` updated to reflect that.

- If a standard structure can be extracted as a subset of the semantics of an extension, the standard tag must be generated along with the extension and kept in sync with it by systems understanding the extension.

> *Example* — If a document has an extension `_LOC` providing a detailed hierarchical place representation with historical names, boundaries, and the like, it must also generate the corresponding `PLAC` (p.82) structures with the subset of that information which `PLAC` can represent.

- If an extension can be extracted as a subset of the semantics of a standard structure, or if the extension and standard structure only sometimes align, then the standard structure should be included if and only if the semantics align in this case.

> *Example* — If a document has an extension `_PARTNER` that generalizes `HUSB` (p.75) and `WIFE` (p.91) and some `ASSO` (p.66) `ROLE` (p.84)s, then it should pair the extension with those standard structures if and only if it knows which one applies.

> *Example* — If a document has an extension `_HOUSEHOLD` that is the same as `FAM` in some situations but not in others, then it should keep the `_HOUSEHOLD` and `FAM` in sync if and only if they align.

- Six standard structure types are exceptions to these rules: `NOTE` (p.80), `SNOTE` (p.85), `INDI.EVEN` (p.71), `FAM.EVEN` (p.71), `INDI.FACT`, and `FAM.FACT`. Each of these allows human-readable text to describe information that cannot be captured in more-specific structures. As such, all other structures express information that could be described using 1 or more of those structure types. Extensions do not need to duplicate their information using any of those structures.

> *Example* — If a document has an extension `_MEMBER` that indicates membership in clubs, boards, and other groups, it is not required to duplicate that information in an `INDI.FACT` because `INDI.FACT` is 1 of the 6 special structure types listed above.

> *Example* — If a document has an extension `_WEIGHT` that describes the weight of a person, it must duplicate that information in an `INDI.DSCR` because `INDI.DSCR` is not 1 of the 6 generic structure types.

1.6. Removing data

There may be situations where data needs to be removed from a dataset, such as when a user requests its deletion or marks it as confidential and not for export.

In general, removed data should result in removed structures.

Pointers to a removed structure should be replaced with `voidPtr` (p.11)s.

If removal of a structure makes the superstructure invalid because the superstructure required the substructure, the structure should instead be retained and have its payload changed to a `voidPtr` if a pointer, or to a data type-appropriate empty value if a non-pointer.

If removing a structure leaves its superstructure with no payload and no substructures, the superstructure should also be removed.

A structure can also be removed if it provides no new information. For example,

```
0 @I1@ INDI
1 NAME John /Doe/
1 NAME John /Doe/
1 FAMC @F1@
1 FAMC @F1@
0 @F1@ FAM
1 CHIL @I1@
1 CHIL @I1@
```

provides no information beyond the simpler form:

```
0 @I1@ INDI
1 NAME John /Doe/
```

2. Data types

Every line value (with any continuation pseudo-structures) is a string. However, those strings can encode 1 of several conceptual data types.

2.1. Text

A free-text string is text in a human language. Conceptually, it may be either a user-generated string or a source-generated string. Programmatically, both are treated as unconstrained sequences of characters with an associated language.

```
anychar = %x09-10FFFF ; but not banned, as with all ABNF rules
Text    = *anychar
```

The URI for the `Text` data type is `xsd:string`.

2.2. Integer

An integer is a non-empty sequence of ASCII decimal digits and represents a non-negative integer in base-10. Leading zeros have no semantic meaning and should be omitted.

```
Integer = 1*DIGIT
```

Negative integers are not supported by this specification.

The URI for the `Integer` data type is `xsd:nonNegativeInteger`.

2.3. Enumeration

An enumeration is a selection from a set of options. They are represented as a string matching the same production as a tag, with the additional permission that standard enumerations may be integers.

```
stdEnum = stdTag / Integer
Enum    = stdEnum / extTag
```

Each structure type with an enumeration payload also defines specific payload values it permits. These permitted payloads match production `stdEnum` and should each have a defined URI. Payload values that match production `extTag` (p.11) are always permitted in structures with an enumeration payload and have their URI defined by the schema.

Each enumeration value has a distinct meaning as identified by its corresponding URI.

The URI of a given tag in an enumeration payload is determined by the tag itself and by the structure type of the structure it is in the payload of.

> *Example* — The tag `HUSB` (p.75) is used in this document to represent two enumeration values. Which one is meant can be identified by the structure type it appears in as follows:
>
> | Containing structure type | Enumeration value identified by tag `HUSB` |
> |---|---|
> | `g7:FAMC-ADOP` (p.65) | `g7:enum-ADOP-HUSB` |
> | `g7:ROLE` (p.84) | `g7:enum-HUSB` |
>
> An extension (p.16) could also place either of these enumeration values in an extension structure type; the extension authors should document which one they permit.
>
> The `HUSB` tag is also used to identify two different structure types, `g7:FAM-HUSB` (p.75) and `g7:HUSB`.

The URI for the `Enum` (p.24) data type is `g7:type-Enum`.

2.4. Date

The date formats defined in this specification include the ability to store approximate dates, date periods, and dates expressed in different calendars.

Technically, there are 3 distinct date data types:

- `DateValue` is a generic type that can express many kinds of dates.
- `DateExact` is used for timestamps and other fully-known dates.
- `DatePeriod` is used to express time intervals that span multiple days.

```
DateValue   = [ date / DatePeriod / dateRange / dateApprox ]
DateExact   = day D month D year  ; in Gregorian calendar
DatePeriod  = [ %s"TO" D date ]
            / %s"FROM" D date [ D %s"TO" D date ]
            ; note both DateValue and DatePeriod can be the empty string

date        = [calendar D] [[day D] month D] year [D epoch]
dateRange   = %s"BET" D date D %s"AND" D date
            / %s"AFT" D date
            / %s"BEF" D date
dateApprox  = (%s"ABT" / %s"CAL" / %s"EST") D date

dateRestrict = %s"FROM" / %s"TO" / %s"BET" / %s"AND" / %s"BEF"
            / %s"AFT" / %s"ABT" / %s"CAL" / %s"EST"

calendar = %s"GREGORIAN" / %s"JULIAN" / %s"FRENCH_R" / %s"HEBREW"
         / extTag

day     = Integer
year    = Integer
month   = stdTag / extTag  ; constrained by calendar
epoch   = %s"BCE" / extTag ; constrained by calendar
```

In addition to the constraints above:

- The allowable `day` (p.25)s, `month`s, `year`s, and `epoch`s are determined by the `calendar`. All known calendars restrict `day` to be between 1 and a month-specific maximum. The largest known maximum is 36, and most months in most calendars have a lower maximum.
- No calendar names, months, or epochs match `dateRestrict`.
- Extension calendars (those with `extTag` for their `calendar`) must use `extTag`, not `stdTag`, for months.

It is recommended that calendars avoid using a single tag to refer to both a month and an epoch.

An absent `calendar` is equivalent to the calendar `GREGORIAN` (p.103).

The grammar above allows for `date`s to be preceded by various words. The meaning of these words is given as follows:

Production	Meaning
`FROM` *x*	Lasted for multiple days, beginning on *x*.
`TO` *x*	Lasted for multiple days, ending on *x*.
`BET` *x*	Exact date unknown, but no earlier than *x*.
`AND` *x*	Exact date unknown, but no later than *x*.

Production	Meaning
`BEF` *x*	Exact date unknown, but no later than *x*.
`AFT` *x*	Exact date unknown, but no earlier than *x*.
`ABT` *x*	Exact date unknown, but near *x*.
`CAL` *x*	*x* is calculated from other data.
`EST` *x*	Exact date unknown, but near *x*; and *x* is calculated from other data.

Known calendars and tips for handling dual dating and extension calendars are given in Appendix A: Calendars and Dates (p.103).

`DateValue` and `DatePeriod` payloads may also be the empty string if no suitable form is known but a substructure (such as a `PHRASE` (p.81) or `TIME` (p.87)) is desired.

> *Note* — Versions 5.3 through 5.5.1 allowed phrases inside `DateValue` payloads. Date phrases were moved to the `PHRASE` substructure in version 7.0. A current limitation, however, is that a phrase in the `PHRASE` substructure cannot specify a language, so if a non-default language is needed to correctly interpret the phrase two options exist:
>
> - `PHRASE` can be used with a documented extension tag for the language, as discussed in `g7:LANG` (p.75).
> - `<<EVENT_DETAIL>>.SOUR.DATA.TEXT` can be used instead along with a `LANG` substructure; this loses the connection with the date, but includes the language with a standard tag.

> *Note* — As defined by the grammar above, every date must have a year. If no year is known, the entire date may be omitted.
>
> > *Example* — The following is an appropriate way to handle a missing year
> >
> > ```
> > 2 DATE
> > 3 PHRASE 5 January (year unknown)
> > ```

The URI for the `DateValue` (p.25) data type is `g7:type-Date`.

The URI for the `DateExact` data type is `g7:type-Date#exact`.

The URI for the `DatePeriod` data type is `g7:type-Date#period`.

2.5. Time

Time is represented on a 24-hour clock (for example, 23:00 rather than 11:00 PM). It may be represented either in event-local time or in Coordinated Universal Time (UTC). UTC is indicated by including a `Z` (U+005A) after the time value; event-local time is indicated by its absence. When a time is used together with a `DateExact`, it is recommended that UTC time be used rather than event-local time.

```
Time     =  hour ":" minute [":" second ["." fraction]] [%s"Z"]

hour     = DIGIT / ("0" / "1") DIGIT / "2" ("0" / "1" / "2" / "3")
minute   = ("0" / "1" / "2" / "3" / "4" / "5") DIGIT
second   = ("0" / "1" / "2" / "3" / "4" / "5") DIGIT
fraction = 1*DIGIT
```

> *Note* — The above grammar prohibits end-of-day instant `24:00:00` and leap-seconds. It allows both `02:50` and `2:50` as the same time.

The URI for the `Time` (p.27) data type is `g7:type-Time`.

2.6. Age

Ages are represented by counts of years, months, weeks, and days.

```
Age         = [[ageBound D] ageDuration]

ageBound    = "<" / ">"
ageDuration = years [D months] [D weeks] [D days]
            / months [D weeks] [D days]
            / weeks [D days]
            / days

years   = Integer %x79    ; 35y
months  = Integer %x6D    ; 11m
weeks   = Integer %x77    ; 8w
days    = Integer %x64    ; 21d
```

Where

Production	Meaning
`<`	The real age was less than the provided age
`>`	The real age was greater than the provided age
`years`	a number of years
`months`	a number of months
`weeks`	a number of weeks
`days`	a number of days

Non-integer numbers should be rounded down to an integer. Thus, if someone has lived for 363.5 days, their age might be written as `363d`, `51w 6d`, `51w`, `0y`, etc.

Because numbers are rounded down, `>` effectively includes its endpoint; that is, the age `> 8d` includes people who have lived 8 days + a few seconds.

Different cultures count ages differently. Some increment years on the anniversary of birth and others at particular seasons. Some round to the nearest year, others round years down, others round years up. Because users may be unaware of these traditions or may fail to convert them to the round-down convention, errors in age of up to a year are common.

> *Note* — Because age payloads are intended to allow recording the age as it was recorded in records that could contain errors, odd ages such as `8w 30d`, `1y 400d`, `1y 30m`, etc. are permitted. Some applications might convert these to more standard forms; if so, it is recommended that they use a `PHRASE` (p.81) substructure to hold the original form.

Age payloads may also be omitted entirely if no suitable form is known but a substructure (such as a `PHRASE`) is desired.

> *Note* — Versions 5.5 and 5.5.1 allowed a few specific phrases inside `Age` (p.28) payloads. Age phrases were moved to the `PHRASE` substructure in 7.0.

The URI for the `Age` data type is `g7:type-Age`.

2.7. List

A list is a meta-syntax representing a sequence of values with another data type. Two list data types are used in this document: List:Text and List:Enum. Lists are serialized in a comma-separated form, delimited by a comma (U+002C `,`) and any number of spaces (U+0020) between each item. It is recommended that a comma-space pair (U+002C U+0020) be used as the delimiter.

```
list      = listItem *(listDelim listItem)
listItem  = [ nocommasp / nocommasp *nocomma nocommasp ]
listDelim = *D "," *D
nocomma   = %x09-2B / %x2D-10FFFF
nocommasp = %x09-1D / %x21-2B / %x2D-10FFFF

List-Text = list
List-Enum = Enum *(listDelim Enum)
```

If valid for the underlying type, empty strings may be included in a list by having no characters between delimiters.

> *Example* — A `List:Text` with value "`, , one, more,`" has 5 `Text` (p.24)-type values: 2 empty strings, the string "`one`", the string "`more`", and 1 more empty string.

There is no escaping mechanism to allow lists of entries that begin or end with spaces or that contain comma characters.

The URI for the `List:Text` data type is `g7:type-List#Text`.

The URI for the `List:Enum` data type is `g7:type-List#Enum`.

2.8. Personal Name

A personal name is mostly free-text. It should be the name as written in the culture of the individual and should not contain line breaks, repeated spaces, or characters not part of the written form of a name (except for U+002F as explained below).

```
PersonalName = nameStr
             / [nameStr] "/" [nameStr] "/" [nameStr]

nameChar     = %x20-2E / %x30-10FFFF  ; any but '/' and '\t'
nameStr      = 1*nameChar
```

The character U+002F (`/`, slash or solidus) has special meaning in a personal name, being used to delimit the portion of the name that most closely matches the concept of a surname, family name, or the like. This specification does not provide any standard way of representing names that contain U+002F.

The URI for the `PersonalName` data type is `g7:type-Name`.

2.9. Language

The language data type represents a human language or family of related languages, as defined in BCP 47. It consists of a sequence of language subtags separated by hyphens, where language subtags are registered by the IANA.

The ABNF grammar for language tags is given in BCP 47, section 2.1, production `Language-Tag`.

The URI for the `Language` data type is `xsd:Language`.

2.10. Media Type

The media type data type represents the encoding of information in bytes or characters, as defined in RFC 2045 and registered by the IANA.

The official grammar for media type is given in RFC 2045, section 5.1, which defines the syntax of registered values and extension values.

```
MediaType = type "/" subtype parameters
```

where:

- `type` and `subtype` are defined in RFC 2045 section 5.1, and registered values (i.e., those not beginning with “x-”) are further constrained by the definitions in RFC 6838, section 4.2. A registry of media types is maintained publicly by the IANA.

- `parameters` is defined in RFC 9110, section 5.6.6. Note that the `parameters` definition in GEDCOM matches that used by HTTP headers which permit whitespace around the ";" delimiter, whereas email headers in RFC 2045 do not.

The URI for the `MediaType` (p.30) data type is `dcat:mediaType`.

2.11. Special

The special data type is a string conforming to a case-specific standard or constraints. The constraints on each special data type instance are either unique to that structure type or are not simply expressed. For example, the payload of an `IDNO` (p.75) structure may obey different rules for each possible `TYPE` (p.90) substructure.

Each special data type is distinct. The URI for the generic data type subsuming all `Special` data types is `xsd:string` (the same as the `Text` (p.24) data type).

```
Special = Text
```

2.12. File Path

The file path data type describes where an digital file is located in a machine-readable way. Syntactically, the payload is a URI reference as defined by RFC 3986, or a valid URL string as defined by the WHATWG URL specification. That is, it can be an absolute or relative URL, optionally with a fragment string.

Version 7.0 only supports the following URLs:

- A URL with scheme `ftp`, `http`, or `https` refers to a **web-accessible file**.
- A URL with scheme `file` refers to either a **local file** or a **non-local file**, as defined by RFC 8089. Local file URLs must not be used in FamilySearch GEDZIP (p.100) and should be avoided in datasets that are expected to be shared on the web or with unknown parties, but may be appropriate for close collaboration between parties with known similar file structures.
- A URI reference with all of the following:
  - no scheme
  - not beginning with `/` (U+002F)
  - not containing any path segments equal to `..` (U+002E U+002E)
  - not containing a reverse solidus character (U+005C `\`) or `banned` (p.9) character, either directly or in escaped form
  - no query or fragment

  refers to a **local file**. If the dataset is part of a GEDZIP file (p.100), the URL of the local file is a zip archive filename; otherwise, the URL of a local file is resolved with *base* equal to the directory containing the dataset.

  It is recommended that local files use the directory prefix `media/`, but doing so is not required.

For compatibility with GEDZIP (p.100) and related formats, it is recommended that the following file paths not be used:

- `gedcom.ged`
- `MANIFEST.MF`
- any URL beginning `META-INF/`

Additional URLs may be supported in future versions of this specification.

The URI for the `FilePath` data type is `g7:type-FilePath`.

3. Genealogical structures

This chapter describes a set of structure types for exchanging family-based lineage-linked genealogical information. Lineage-linked data pertains to individuals linked in family relationships across multiple generations.

The genealogical structures defined in this chapter are based on the general framework of the container format and data types defined in Chapters 1 and 2.

Historically, these genealogical structures were used as the only form approved for exchanging data with Ancestral File, TempleReady and other Family History resource files. Those systems were all replaced between 1999 and 2019, and GEDCOM-X was introduced as the new syntax for communication with their replacements. FamilySearch GEDCOM 7.0 and GEDCOM-X have similar expressive power, but as of 2021 GEDCOM is more common for exchanging single-researcher files between applications and GEDCOM-X is more common for transferring bulk data and communication directly between applications.

The basic description of the genealogical structures' organization is presented in the following 3 major sections:

- "Structure Organization (p.35)" describes records and other nested structures.
- "Structure Meaning (p.59)" provides a definition of each structure by its tag.
- "Enumeration Values (p.92)" provides a definition of each enumeration value by its containing structure.

3.1. A Metasyntax for Structure Organization

The structures, with their payloads and substructures, are represented using a custom metasyntax. The intent of this metasyntax is to resemble the line encoding of allowable structures. In the metasyntax:

- Options are placed between brackets `[` and `]` and have choices separated by pipes `|`.
- Named sets of rules are indicated with a name followed by `:=`.
- Level markers are used to indicate substructure relationships.
  - `0` means "must be a record".
  - `n` means "level inherited from rule instantiation".
  - `+1`, `+2`, etc., indicate nesting within nearest preceding structure with lesser level.
- Four cardinality markers are used: `{0:1}`, `{1:1}`, `{0:M}`, and `{1:M}`.
  - `{0:` means "optional" – the structure may be omitted
  - `{1:` means "required" – at least 1 must appear
  - `:M}` means "any number" – 1 or more structures may appear. Unless otherwise specified, the first is the most-preferred value. If an application needs to display just 1 of several

NAME (p.79)s, BIRT (p.67)s, etc, they should show the first such structure unless more specific selection criteria are available.
  - :1} means "singular" – at most 1 may appear; a second must not be present.

Systems interested in violating the cardinality rules should instead create extension structures (p.16) with different cardinality.

- Rule instantiation is indicated by the rule name in double angle-brackets (such as <<rule name>>) and a cardinality marker.

  The cardinality markers of rule instantiations and their referenced line templates are combined such that the resulting cardinality is required only if both combined cardinalities are required and singular only if both combined cardinalities are singular.

> *Example* — The definition of the FAM record has the line
>
> ```
> +1 <<CREATION_DATE>>                    {0:1}
> ```
>
> and the CREATION_DATE (p.45) rule begins
>
> ```
> n CREA                                  {1:1}  g7:CREA
> ```
>
> Thus, a FAM record has an optional singular CREA (p.68) substructure (such as cardinality {0:1}).

- Line templates have several parts:
  - An optional cross-reference template @XREF:tag@, meaning this structure may be pointed to by other structures.

    Structures that are not pointed to by other structures need not have a cross-reference identifier (p.11) even if their line template has a cross-reference template.
  - The standard tag for this structure.
  - An optional payload descriptor; if present this is 1 of the following:
    - @<XREF:tag>@ means a pointer to a structure with this cross-reference template; @VOID@ is also permitted.
    - <data type> means a non-pointer payload, as described in Data types (p.24). If the data type allows the empty string, the payload may be omitted.
    - [text|<NULL>] means the payload is optional but if present must be the given text.

    If there is a payload descriptor, a payload that matches the payload is required of the described structure unless the descriptor says the payload is optional.

    If there is no payload descriptor, the described structure must not have a payload.
  - A cardinality marker.
  - The URI of this structure type.

    Pseudo-structures do not have a URI.

- Within the metasyntax, the order in which substructures are presented within a structure and the order in which choices are presented within an option set are not significant unless otherwise specified in the text next to the metasyntax block.

The context of a structure's superstructure may be necessary in addition to the structure's standard tag to fully determine its structure type. To refer to a structure in the context of its superstructure, tags are written with intervening periods. For example, `GEDC.VERS` (p.91) refers to a structure with tag `VERS` (p.91) and a superstructure with tag `GEDC` (p.74).

3.2. Structure Organization

3.2.1. Document

Dataset :=

```
0 <<HEADER>>                                  {1:1}
0 <<RECORD>>                                  {0:M}
0 TRLR                                        {1:1}  g7:TRLR
```

The order of these is significant: the `HEADER` (p.36) must come first and `TRLR` (p.90) must be last, with any `RECORD`s in between.

`RECORD` :=

```
[
n <<FAMILY_RECORD>>                           {1:1}
|
n <<INDIVIDUAL_RECORD>>                       {1:1}
|
n <<MULTIMEDIA_RECORD>>                       {1:1}
|
n <<REPOSITORY_RECORD>>                       {1:1}
|
n <<SHARED_NOTE_RECORD>>                      {1:1}
|
n <<SOURCE_RECORD>>                           {1:1}
|
n <<SUBMITTER_RECORD>>                        {1:1}
]
```

HEADER :=

```
n HEAD                                    {1:1}  g7:HEAD
  +1 GEDC                                 {1:1}  g7:GEDC
     +2 VERS <Special>                    {1:1}  g7:GEDC-VERS
  +1 SCHMA                                {0:1}  g7:SCHMA
     +2 TAG <Special>                     {0:M}  g7:TAG
  +1 SOUR <Special>                       {0:1}  g7:HEAD-SOUR
     +2 VERS <Special>                    {0:1}  g7:VERS
     +2 NAME <Text>                       {0:1}  g7:NAME
     +2 CORP <Text>                       {0:1}  g7:CORP
        +3 <<ADDRESS_STRUCTURE>>          {0:1}
        +3 PHON <Special>                 {0:M}  g7:PHON
        +3 EMAIL <Special>                {0:M}  g7:EMAIL
        +3 FAX <Special>                  {0:M}  g7:FAX
        +3 WWW <Special>                  {0:M}  g7:WWW
     +2 DATA <Text>                       {0:1}  g7:HEAD-SOUR-DATA
        +3 DATE <DateExact>               {0:1}  g7:DATE-exact
           +4 TIME <Time>                 {0:1}  g7:TIME
        +3 COPR <Text>                    {0:1}  g7:COPR
  +1 DEST <Special>                       {0:1}  g7:DEST
  +1 DATE <DateExact>                     {0:1}  g7:HEAD-DATE
     +2 TIME <Time>                       {0:1}  g7:TIME
  +1 SUBM @<XREF:SUBM>@                   {0:1}  g7:SUBM
  +1 COPR <Text>                          {0:1}  g7:COPR
  +1 LANG <Language>                      {0:1}  g7:HEAD-LANG
  +1 PLAC                                 {0:1}  g7:HEAD-PLAC
     +2 FORM <List:Text>                  {1:1}  g7:HEAD-PLAC-FORM
  +1 <<NOTE_STRUCTURE>>                   {0:1}
```

The header pseudo-structure provides metadata about the entire dataset. A few substructures of note:

- `GEDC` (p.74) identifies the specification that this document conforms to. It is recommended that `GEDC` be the first substructure of the header.
- `SCHMA` (p.85) gives the meaning of extension tags; see Extensions (p.16) for more details.
- `SOUR` (p.85) describes the originating software.
  - `CORP` (p.68) describes the corporation creating the software.
  - `HEAD.SOUR.DATA` (p.69) describes a larger database, electronic data source, or digital repository this data is extracted from.
- `LANG` (p.75) and `PLAC` (p.82) give a default value for the rest of the document.

> *Deprecation Note* — `HEAD.SOUR.DATA` is now deprecated and applications should use `HEAD.SOUR.NAME` instead.

3.2.2. Records

FAMILY_RECORD :=

```
n @XREF:FAM@ FAM                          {1:1}  g7:record-FAM
  +1 RESN <List:Enum>                     {0:1}  g7:RESN
  +1 <<FAMILY_ATTRIBUTE_STRUCTURE>>       {0:M}
  +1 <<FAMILY_EVENT_STRUCTURE>>           {0:M}
  +1 <<NON_EVENT_STRUCTURE>>              {0:M}
  +1 HUSB @<XREF:INDI>@                   {0:1}  g7:FAM-HUSB
     +2 PHRASE <Text>                     {0:1}  g7:PHRASE
  +1 WIFE @<XREF:INDI>@                   {0:1}  g7:FAM-WIFE
     +2 PHRASE <Text>                     {0:1}  g7:PHRASE
  +1 CHIL @<XREF:INDI>@                   {0:M}  g7:CHIL
     +2 PHRASE <Text>                     {0:1}  g7:PHRASE
  +1 <<ASSOCIATION_STRUCTURE>>            {0:M}
  +1 SUBM @<XREF:SUBM>@                   {0:M}  g7:SUBM
  +1 <<LDS_SPOUSE_SEALING>>               {0:M}
  +1 <<IDENTIFIER_STRUCTURE>>             {0:M}
  +1 <<NOTE_STRUCTURE>>                   {0:M}
  +1 <<SOURCE_CITATION>>                  {0:M}
  +1 <<MULTIMEDIA_LINK>>                  {0:M}
  +1 <<CHANGE_DATE>>                      {0:1}
  +1 <<CREATION_DATE>>                    {0:1}
```

The `FAM` record was originally structured to represent families where a male `HUSB` (p.75) (husband or father) and female `WIFE` (p.91) (wife or mother) produce `CHIL` (p.68) (children). The `FAM` record may also be used for cultural parallels to this, including nuclear families, marriage, cohabitation, fostering, adoption, and so on, regardless of the gender of the partners. Sex, gender, titles, and roles of partners should not be inferred based on the partner that the `HUSB` or `WIFE` structure points to.

The individuals pointed to by the `HUSB` and `WIFE` are collectively referred to as “partners”, “parents” or “spouses”.

Some displays may be unable to display more than 2 partners. Displays may use `HUSB` and `WIFE` as layout hints, for example, by consistently displaying the `HUSB` on the same side of the `WIFE` in a tree view. Family structures with more than 2 partners may either use several `FAM` records or use `ASSOCIATION_STRUCTURE` (p.44)s to indicate additional partners. `ASSO` (p.66) should not be used for relationships that can be expressed using `HUSB`, `WIFE`, or `CHIL` instead.

> *Note* — The `FAM` record will be revised in a future version to more fully express the diversity of human family relationships.

The order of the `CHIL` (children) pointers within a `FAM` (family) structure should be chronological by birth; this is an exception to the usual "most preferred value first" rule. A `CHIL` with a `voidPtr` (p. 11) indicates a placeholder for an unknown child in this birth order.

If a `FAM` record uses `HUSB` or `WIFE` to point to an `INDI` record, the `INDI` record must use `FAMS` (p.73) to point to the `FAM` record. If a `FAM` record uses `CHIL` to point to an `INDI` record, the `INDI` record must use a `FAMC` (p.73) to point to the `FAM` record.

An `INDI` record should not have multiple `FAMS` substructures pointing to the same `FAM`.

A `FAM` record should not have multiple `CHIL` substructures pointing to the same `INDI`; doing so implies a nonsensical birth order. An `INDI` record may have multiple `FAMC` substructures pointing to the same `FAM`, but doing so is not recommended.

Source citations and notes related to the start of a specific child relationship should be placed under the child's `BIRT` (p.67), `CHR` (p.68), or `ADOP` (p.65) event, rather than under the `FAM` record.

`INDIVIDUAL_RECORD` :=

```
n @XREF:INDI@ INDI                          {1:1}  g7:record-INDI
  +1 RESN <List:Enum>                       {0:1}  g7:RESN
  +1 <<PERSONAL_NAME_STRUCTURE>>            {0:M}
  +1 SEX <Enum>                             {0:1}  g7:SEX
  +1 <<INDIVIDUAL_ATTRIBUTE_STRUCTURE>>     {0:M}
  +1 <<INDIVIDUAL_EVENT_STRUCTURE>>         {0:M}
  +1 <<NON_EVENT_STRUCTURE>>                {0:M}
  +1 <<LDS_INDIVIDUAL_ORDINANCE>>           {0:M}
  +1 FAMC @<XREF:FAM>@                      {0:M}  g7:INDI-FAMC
     +2 PEDI <Enum>                         {0:1}  g7:PEDI
        +3 PHRASE <Text>                    {0:1}  g7:PHRASE
     +2 STAT <Enum>                         {0:1}  g7:FAMC-STAT
        +3 PHRASE <Text>                    {0:1}  g7:PHRASE
     +2 <<NOTE_STRUCTURE>>                  {0:M}
  +1 FAMS @<XREF:FAM>@                      {0:M}  g7:FAMS
     +2 <<NOTE_STRUCTURE>>                  {0:M}
  +1 SUBM @<XREF:SUBM>@                     {0:M}  g7:SUBM
  +1 <<ASSOCIATION_STRUCTURE>>              {0:M}
  +1 ALIA @<XREF:INDI>@                     {0:M}  g7:ALIA
     +2 PHRASE <Text>                       {0:1}  g7:PHRASE
  +1 ANCI @<XREF:SUBM>@                     {0:M}  g7:ANCI
  +1 DESI @<XREF:SUBM>@                     {0:M}  g7:DESI
  +1 <<IDENTIFIER_STRUCTURE>>               {0:M}
  +1 <<NOTE_STRUCTURE>>                     {0:M}
  +1 <<SOURCE_CITATION>>                    {0:M}
  +1 <<MULTIMEDIA_LINK>>                    {0:M}
  +1 <<CHANGE_DATE>>                        {0:1}
  +1 <<CREATION_DATE>>                      {0:1}
```

The individual record is a compilation of facts or hypothesized facts about an individual. These facts may come from multiple sources. Source citations and notes allow documentation of the source where each of the facts were discovered.

A single individual may have facts distributed across multiple individual records, connected by `ALIA` (p.66) (alias, in the computing sense not the pseudonym sense) pointers. See `ALIA` for more details.

Individual records are linked to Family records by use of bi-directional pointers. Details about those links are stored as substructures of the pointers in the individual record. Source citations and notes related to the start of the individual's relationship to parents should be placed under the individual's `BIRT`, `CHR` (p.68), or `ADOP` (p.65) event, rather than directly under the `INDI` record, since the former permits explicitly identifying the family record whereas the latter does not.

Other associations or relationships are represented by the `ASSO` (p.66) (association) tag. The person's relation or associate is the person being pointed to. The association or relationship is stated by the value on the subordinate `ROLE` (p.84) line. `ASSO` should not be used for relationships that can be expressed using `FAMS` (p.73) or `FAMC` (p.73) instead.

> *Example* — The following example refers to 2 individuals, `@I1@` and `@I2@`, where `@I2@` is a godparent of `@I1@`:
>
> ```
> 0 @I1@ INDI
> 1 ASSO @I2@
> 2 ROLE GODP
> ```

Events stored as facts within an `INDI` record may also have `FAMC` or `ASSO` tags to indicate families and individuals that participated in those events. For example, a `FAMC` pointer subordinate to an adoption event indicates a relationship to family by adoption; biological parents can be shown by a `FAMC` pointer subordinate to the birth event; the eulogist at a funeral can be shown by an `ASSO` pointer subordinate to the burial event; and so on. A subordinate `FAMC` pointer is allowed to refer to a family where the individual does not appear as a child.

MULTIMEDIA_RECORD :=

```
n @XREF:OBJE@ OBJE                          {1:1}  g7:record-OBJE
  +1 RESN <List:Enum>                       {0:1}  g7:RESN
  +1 FILE <FilePath>                        {1:M}  g7:FILE
     +2 FORM <MediaType>                    {1:1}  g7:FORM
        +3 MEDI <Enum>                      {0:1}  g7:MEDI
           +4 PHRASE <Text>                 {0:1}  g7:PHRASE
     +2 TITL <Text>                         {0:1}  g7:TITL
     +2 TRAN <FilePath>                     {0:M}  g7:FILE-TRAN
        +3 FORM <MediaType>                 {1:1}  g7:FORM
  +1 <<IDENTIFIER_STRUCTURE>>               {0:M}
  +1 <<NOTE_STRUCTURE>>                     {0:M}
  +1 <<SOURCE_CITATION>>                    {0:M}
  +1 <<CHANGE_DATE>>                        {0:1}
  +1 <<CREATION_DATE>>                      {0:1}
```

The multimedia record refers to 1 or more external digital files, and may provide some additional information about the files and the media they encode.

The file reference can occur more than once to group multiple files together. Grouped files should each pertain to the same context. For example, a sound clip and a photo both of the same event might be grouped in a single `OBJE` (p.80).

The change and creation dates should be for the `OBJE` record itself, not the underlying files.

REPOSITORY_RECORD :=

```
n @XREF:REPO@ REPO                          {1:1}  g7:record-REPO
  +1 NAME <Text>                            {1:1}  g7:NAME
  +1 <<ADDRESS_STRUCTURE>>                  {0:1}
  +1 PHON <Special>                         {0:M}  g7:PHON
  +1 EMAIL <Special>                        {0:M}  g7:EMAIL
  +1 FAX <Special>                          {0:M}  g7:FAX
  +1 WWW <Special>                          {0:M}  g7:WWW
  +1 <<NOTE_STRUCTURE>>                     {0:M}
  +1 <<IDENTIFIER_STRUCTURE>>               {0:M}
  +1 <<CHANGE_DATE>>                        {0:1}
  +1 <<CREATION_DATE>>                      {0:1}
```

The repository record provides information about an institution or person that has a collection of sources. Informal repositories include the owner of an unpublished work or of a rare published source, or a keeper of personal collections. An example would be the owner of a family Bible containing unpublished family genealogical entries.